

Second Lab - Program 0.2.5

Your next enhancement to the project is robust unit testing. Currently you only test the functional behaviors. Now, we want to add wider system tests. When functions have multiple behaviors, test each of them in their own `test` function rather than create one uber-test that checks all the behaviors. For example, `process_file` has an error reporting behavior and a functional behavior that is dependent on a file read. These two behaviors should get their own tests.

Use the `pytest` options discussed further below to simplify running and checking individual tests and to evaluate the coverage of your tests. **Do a coverage check before you start your work and then keep checking as you work. Your coverage should improve and the missing lines, if they exist, should make sense (not tested yet or can justify not testing).**

Behaviors To Test

Your program utilizes a number of things that require more sophisticated testing than what we've done previously:

- calls `sys.exit` are raising `SystemExit`
- output to `stderr` and `stdout` should be checked
- logging should be checked without adding entries to actual logs
- mock `datetime.now` to avoid time-dependent errors in tests
- mock `pyhop.traceroute` to get predictable example routes.
- use `tmp_path` to test file I/O without actually generating files.
- (Optional) use fixtures to mock CLI arguments and run a whole system test (maybe with more mocks...)

0.2.5 - To Do

1. `process_file` should produce output on `stderr` and raise `SystemExit` when there's a problem opening the file. Write a test to verify that your error message is, in fact, reported to the user and when it is the system exit method is invoked.
2. `process_file`'s functional behavior depends on reading a file. Use `pytest`'s temporary directory and file features to create unit test for this behavior that does not rely on previously existing files and directories.
3. `do_trace` has a functional behavior dependent on a system call to `traceroute`. Use monkeypatching to patch the behavior of `pyhop.traceroute` and write a test to verify `do_trace`'s expected functionality. You'll probably also need to monkeypatch some `datetime` code as well.
4. `do_trace` creates log entries which are also dependent on the system call. Use monkeypatching and `pytest`'s `caplog` fixture to test your expected logging behaviors.
5. `all_to_csv`'s primary behavior is to write to a file. Use `pytest`'s temporary directory and file features to create unit test for this behavior that does not rely on previously existing files and directories.
6. `all_to_csv` also writes log data. Once again, use `caplog` to test your logging behaviors.

When you're done, update your application version number in `pyproject.toml` and sync your new code.

Pytest Options

When you do `uv run pytest` it will run all your tests. It is often really helpful to just run individual tests. To do this you need to know the *path* to the file containing your tests and the name of the test function you wish to run. You can then call the individual test. If your tests are in `test_mod.py` which is located in the directory `tests`, and you wish to run the tests named `test_func`, then you'd do the following:

```
uv run pytest tests/test_mod.py::test_func
```

That's `uv run pytest PATH::TEST_NAME`.

To run your tests with a coverage report you need to specify the *package* you want checked for coverage. You then add the `--cov=PACKAGE_NAME` option to your pytest invocation. Be aware that `*pytest-cov` will only evaluate coverage at the level of a file or module, not an individual function. To get a report on what percent of the module `mod` (`mod.py`) are covered by the tests in `tests/`, you would run:

```
uv run pytest --cov=mod tests/
```

If you want the line numbers for the code that's not covered you can use `--cov-report=term-missing` as well:

```
uv run pytest --cov=mod --cov-report=term-missing tests/
```